

Automatic Music Genre Classification

DEEP LEARNING — FINAL PROJECT · UPF 2026

Núria Esquius Bau and Berta Miguel Ninou

Why classify music genres automatically?

Streaming platforms need to organize millions of tracks automatically

Manual labeling is slow, expensive, and subjective

Deep learning can detect patterns in audio structure, rhythm, and timbre

- Genre is a useful proxy for recommendation and content filtering
- GTZAN provides a standard benchmark to compare approaches [1].



GTZAN dataset

Total

1,000

Classes

10

Train set

800

Features

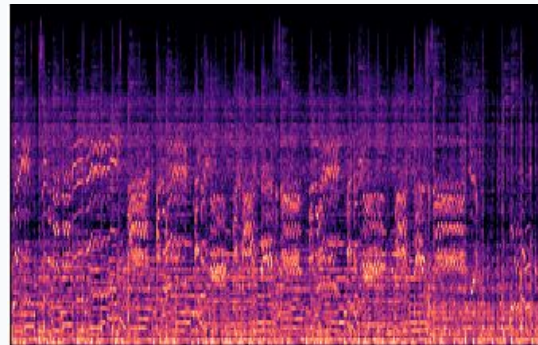
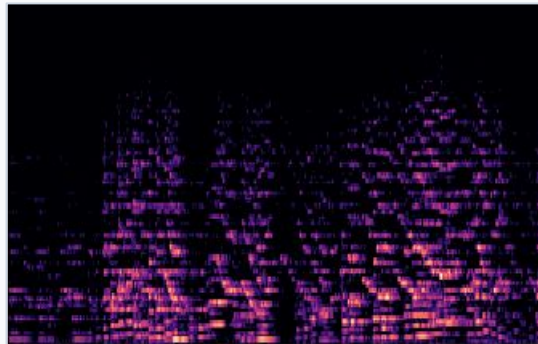
57

1,000 audio clips — 100 per genre, 10 genres

- Each clip: 30 seconds of music

- Mel Spectrogram:** One for each audio clip
- 57 Features from the audio file:** split audios in 3 seconds → increases x10 data

Main challenge: only 800 training samples after 80/10/10 split



Filename	length	chroma_stft_mean	chroma_stft_var	rms_mean	rms_var	spectral_centroid...	spectral_centroid...	spectral_bandwi...	spectral_bandwi...
9990 unique values	66.1k	0.31	0.02	0	0	473	812	499	138k
blues.00000.0.wav	66149	0.3354063630184065	0.09184029201568527	0.1304050236940384	0.0035210042260587215	1773.8650319904662	167541.6308606573	1972.7443081356735	117335.7715632089
blues.00000.1.wav	66149	0.3438653512477875	0.0861465260386467	0.1126992478966713	0.001496854273682366	1816.6937765281948	90525.69086581325	2018.851080132973	65671.87567304078
blues.00000.2.wav	66149	0.34681475162506104	0.092242891891627365	0.132089338184833527	0.0042620399326086044	1788.539718722745	111407.43761296556	2084.5651317440537	75124.92171573533
blues.00000.3.wav	66149	0.3636367884616852	0.08685615656760071	0.1325647234916687	0.0024475634563714266	1655.289045406881	111952.2845173748	1968.8399882077425	82913.6392600478
blues.00000.4.wav	66149	0.33557942509651184	0.08812054439020157	0.1432080800001818	0.001700080175967753	1636.6561993518775	79667.26765440499	1948.5038044114322	60204.02026023243
blues.00000.5.wav	66149	0.3766697347164154	0.08970216701227188	0.1326178014278412	0.0035825634840875064	1994.9152192705897	211700.61956933062	2152.7678542146077	74263.87318240977
blues.00000.6.wav	66149	0.3799807405204773	0.0888271199264526	0.130347796201706	0.0031602711407089264	1962.1508958726688	177443.07004540853	2146.503479192859	98020.54142181856
blues.00000.7.wav	66149	0.33187994360923767	0.09211885184049006	0.140600323677063	0.002545942086726427	1701.0909235701698	35678.13061562547	1979.3076121320902	36670.725886003514
blues.00000.8.wav	66149	0.3478773032321167	0.09420917183160782	0.13312990963459015	0.002538164146244526	1746.4735023276876	138073.93124392346	1087.619723142195	117069.92004901407
blues.00000.9.wav	66149	0.3508612540245856	0.08295709639707674	0.11531206220308412	0.001846020924850749	1763.9409423059536	61493.42312137413	1074.1957101296146	51944.921435060005
blues.00001.0.wav	66149	0.4024008010520172	0.090302367260091782	0.09302367269992028	0.003075555470585823	1279.1822416109314	406513.8167436446	1921.306191551141	196573.44164008124

The classification problem

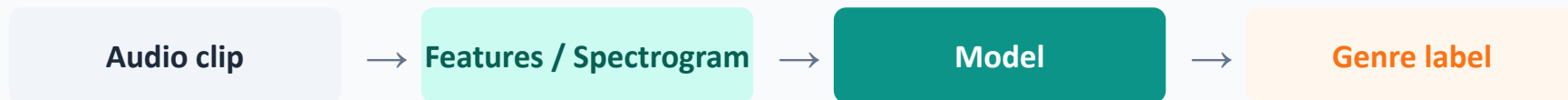
Input: 30-second audio clips

- Try 3 different models

Output: one of 10 genre classes - multi-class classification

- Classes: blues, classical, country, disco, hiphop, jazz, metal, pop, reggae, rock

Metric: accuracy



Preprocessing the data

Spectrograms (CNN): resized to 128×128 · normalized with mean/std [0.5, 0.5, 0.5]

Spectrograms (ResNet): resized to 224×224 · ImageNet normalization [2,3]

Tabular features: StandardScaler → zero mean, unit variance

- Split: 80% train / 10% val / 10% test — same indices across all models

No flipping/jitter: spectrogram axes encode time and frequency — distorting them breaks physical meaning



DNN on tabular audio features

Input: 57 hand-crafted features (MFCCs, chroma, tempo...)

- 3-layer MLP: $57 \rightarrow 256 \rightarrow 128 \rightarrow 10$ with ReLU + Dropout
 - **Optimizer:** Adam lr=1e-3 [5]
 - **Loss:** CrossEntropyLoss
 - **100 epochs**
- Most stable and efficient model due to compact, meaningful input
- StratifiedKfold

57 features



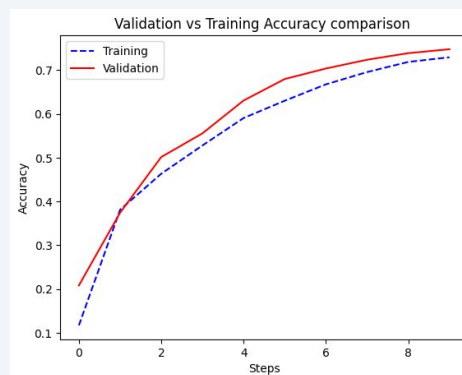
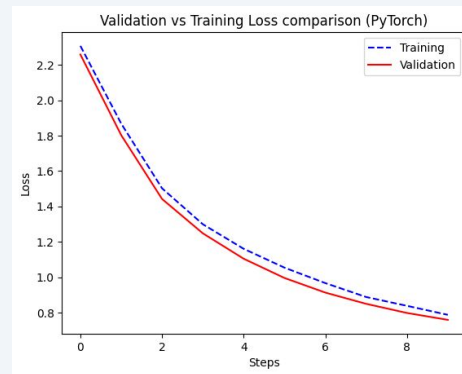
Linear 256



Linear 128



10 classes



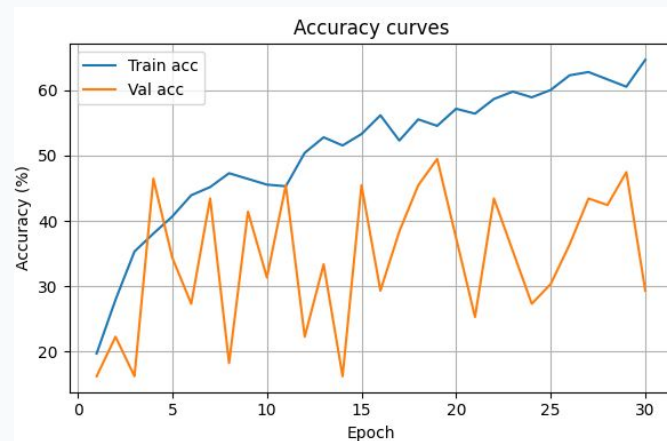
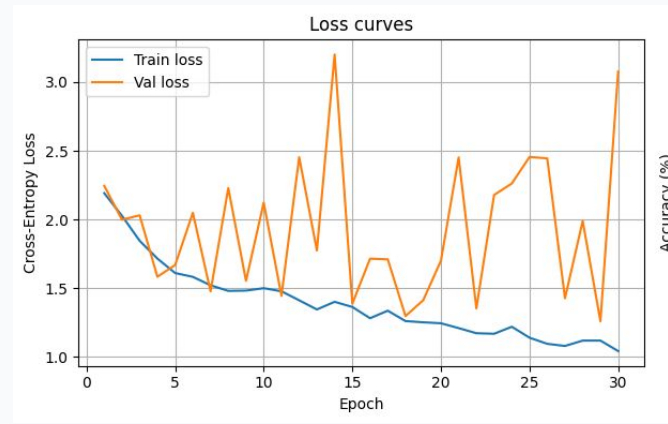
CNN on spectrograms

Input: 128×128 spectrogram images (3 channels)

Learns spatial time-frequency patterns directly from pixels

- 3 conv blocks: 32→64→128 filters · BatchNorm + ReLU + MaxPool
- AdaptiveAvgPool → Dropout → Linear(128→10)

Optimizer: Adam lr=1e-3, 30 epochs



Transfer learning with ResNet

ResNet: pretrained on ImageNet – 11M parameters, knows edges, textures, patterns,...

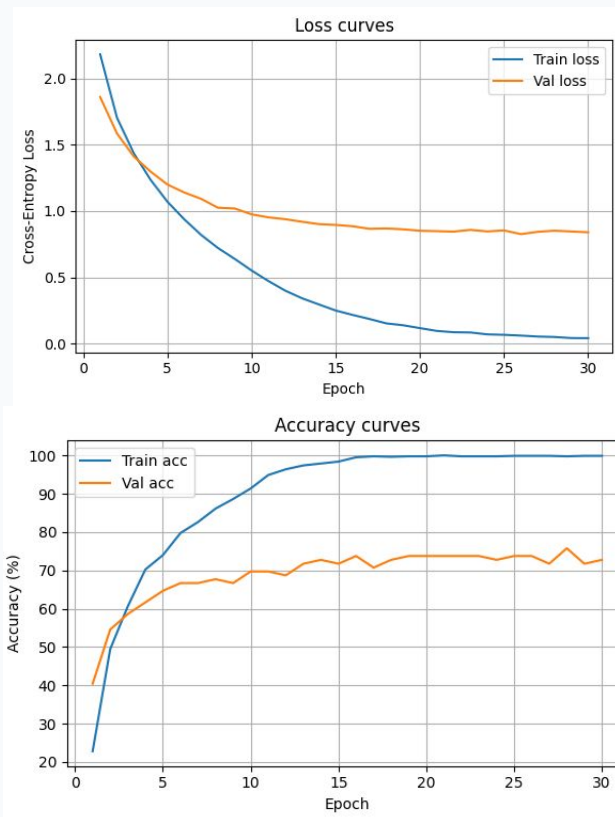
- Input: 224×224 with ImageNet normalization [2,3]

4 fine-tuning strategies:

1. fully connected (fc) only
2. layer4+fc
3. layer3+layer4+fc
4. layer2+layer3+layer4+fc

- Gradual unfreezing: deeper layers use $lr=1e-5$, fc uses $lr=1e-4$

Key advantage: effective even with small datasets, doesn't relearn from scratch



Experimental setup – all models

Model	Input	Architecture	Epochs	Optimizer
DNN	57 features	3-layer MLP	100	Adam 1e-3
CNN	128×128 spec.	3 conv blocks	40	Adam 1e-3
ResNet — fc	224×224 spec.	ResNet18, head only	30	Adam 1e-3
ResNet — L4+fc	224×224 spec.	ResNet18, layer4+fc	30	Adam 1e-5/1e-4
ResNet — L3+4+fc	224×224 spec.	ResNet18, layer3+4+fc	30	Adam 1e-5/1e-4
ResNet — L2+3+4+fc	224×224 spec.	ResNet18, layer2+3+4+fc	30	Adam 1e-5/1e-4

Test accuracy — all models

Why do these results make sense?

CNN instability: 800 images is too few for a 3-block CNN → overfits

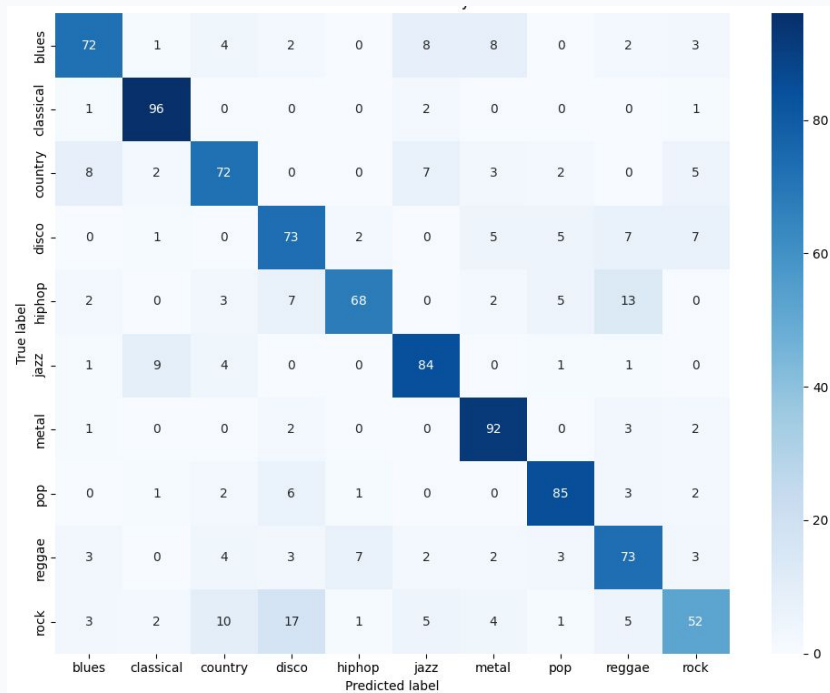
Transfer learning:

- **L3 and L4 better:** deeper features adapt to spectrogram structure
- **L2:** Already captures useful generic features

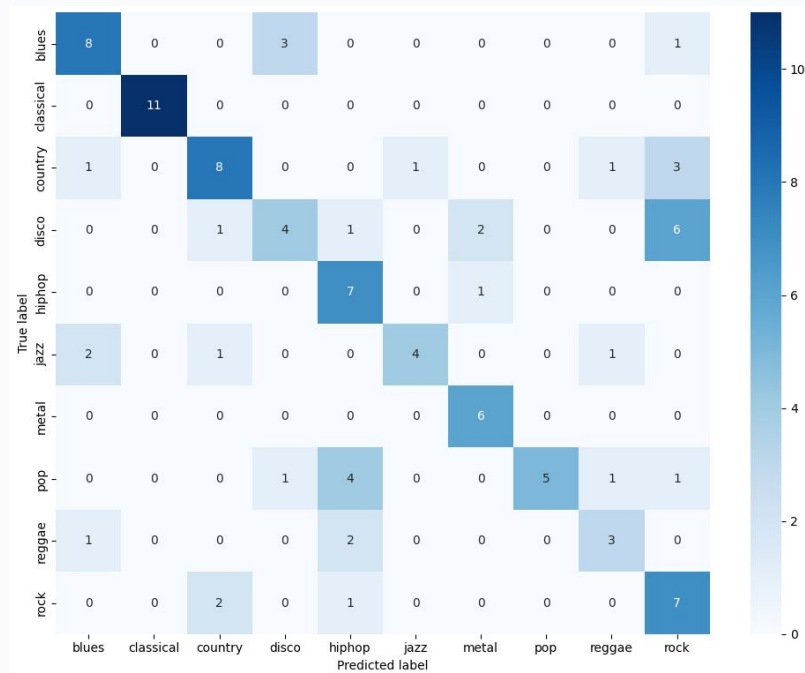
DNN competitive: pre-extracted and more features → simpler problem for MLP

Model	Test acc.
DNN	76.18%
DNN (5-fold)	76.48% +/- 0.01
CNN (scratch)	~60%
ResNet — fc	62.38%
ResNet — L4+fc	65.35%
ResNet — L3+L4+fc	72.28%
ResNet — L2+L3+L4+fc	68.32%

Confusion matrices

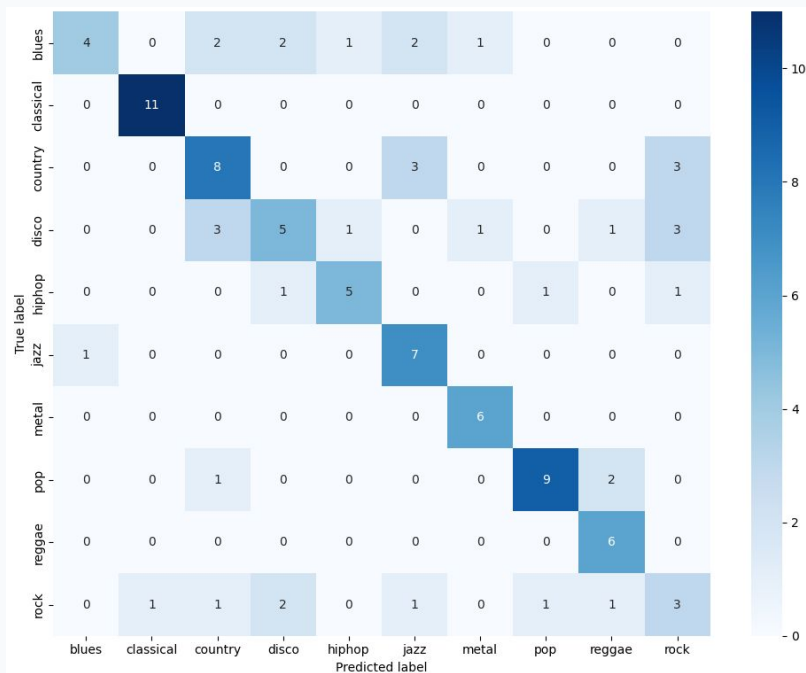


DNN

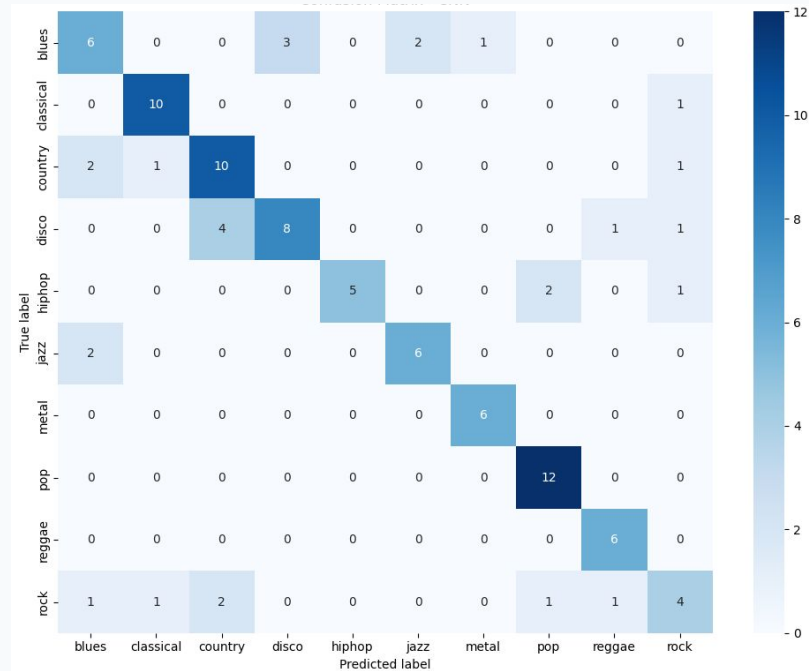


CNN

Confusion matrices



ResNet - fc



ResNet - layer3-4-fc

Conclusions

BEST RESULT

DNN (57 features)
76.18% test accuracy

CNN FROM SCRATCH

Unstable due to small
dataset (800 samples)

TRANSFER LEARNING

Effective with small datasets
Doesn't relearn from scratch

FUTURE WORK

LR scheduler · Soft voting

References

1. Andradaolteanu. (2020, March 23). GTZAN Dataset - Music Genre Classification. Kaggle.
<https://www.kaggle.com/datasets/andradaolteanu/gtzan-dataset-music-genre-classification>
2. Rosebrock, A. (2017, March 19). ImageNet: VGGNet, ResNet, Inception, and Xception with Keras. PyImageSearch.
<https://pyimagesearch.com/2017/03/20/imagenet-vggnet-resnet-inception-xception-keras/>
3. GeeksforGeeks. (2021, April 15). How to normalize images in PyTorch? GeeksforGeeks.
<https://www.geeksforgeeks.org/python/how-to-normalize-images-in-pytorch/>
4. PyTorch. (n.d.). torch.nn — PyTorch 2.12 documentation.
<https://docs.pytorch.org/docs/2.12/nn.html>
5. GeeksforGeeks. (2020, October 21). Introduction To Adam Optimizer.
<https://www.geeksforgeeks.org/deep-learning/adam-optimizer/>
6. Deep Learning Course, Class 9: Convolutional Neural Networks [PDF]. UPF, 2026.
<https://www.kaggle.com/datasets/andradaolteanu/gtzan-dataset-music-genre-classification>

Thank you!

Questions?

DEEP LEARNING — FINAL PROJECT · UPF 2026

Núria Esquiús Bau and Berta Miguel Ninou